BACKEND NOTES

HTTP, Servers, Ports & Express.js

From writing your very first server to building a complete CRUD API — explained in plain, simple language.

Client & Server | HTTP / HTTPS | Ports & listen() | Raw http module | Express.js | CRUD APIs | Middleware | express.json() | params / query / body / files

01 First things first — what is backend?

When you open Instagram, you see photos, likes and comments. None of that is already stored on your phone. Your phone **asks another computer** for it, and that computer **sends it back**.

Client

The one who **asks**. Your browser, a mobile app, or a tool like Postman.

Server

The one who **gives**. A program that stays switched on and keeps listening.

Backend is the code that runs on the server. That is exactly what you are learning to write now.

> **The apartment building analogy — remember this one**
>
> It will come back again and again throughout these notes.

REAL WORLD	TECH WORLD
Address of a building	IP address
Flat number inside that building	Port
The person sitting inside that flat	Server program
Sitting near the door, waiting for the bell	`listen()`
Ringing the bell and asking for something	Request
The person handing the thing back to you	Response
The common language both of you speak	HTTP
Talking through a sealed envelope	HTTPS

02 What is HTTP?

HTTP stands for **H**yper**T**ext **T**ransfer **P**rotocol.

The word **protocol** simply means **a set of rules**.

Imagine walking into a shop and saying, “One packet of biscuits, please.” The shopkeeper understands you because both of you speak the same language. If you said it in Japanese, nothing would happen.

In exactly the same way, the client and the server need one **fixed language** to talk in. That language is HTTP.

The rules of HTTP decide:

- What format a request must be sent in
- What format the response will come back in
- Which action uses which method — GET, POST, PUT, DELETE
- How the server tells you whether the job succeeded — the status code

> **HTTP is stateless**
>
> The server **does not remember** what you asked for last time. Every request is treated as a completely fresh conversation. That is why you have to send your identity (a token) along with every single request. This will make much more sense when you reach the login and JWT topic.

HTTP vs HTTPS

HTTPS = HTTP + Secure

HTTP

Data travels as **plain text**. Anyone sitting in the middle — a hacker, or the owner of the WiFi you are using — can read your password clearly.

HTTPS

The same data travels **encrypted**, that is, locked. Anyone in the middle sees only meaningless symbols.

Analogy: HTTP is a **postcard** — whoever handles it can read it. HTTPS is a **sealed and locked envelope** — only the intended person can open it.

Every real website today runs on HTTPS. While developing on your own laptop (localhost), plain HTTP is perfectly fine, so do not worry about it yet.

03 Request and response — the full cycle

A request has 4 parts

1. Method — what to do (GET / POST / PUT / DELETE)
2. URL — on which resource (`/users`, `/users/5`)
3. Headers — extra information (token, content-type)
4. Body — the actual data (only in POST and PUT)

A response has 3 parts

1. Status code — did the job succeed or not
2. Headers — information about the response
3. Body — the actual data (JSON or HTML)

Status codes — an easy way to remember them

RANGE	MEANING	COMMON CODES
`2xx`	Everything went fine	200 OK · 201 Created
`3xx`	Go somewhere else	301 · 302 Redirect
`4xx`	The **client** made a mistake	400 Bad Request · 401 Unauthorized · 404 Not Found
`5xx`	The **server** made a mistake	500 Internal Server Error

> **Remember it in one line**
>
> **4xx means you messed up. 5xx means I messed up.**

04 What is a port, and why do we need it?

Many programs can run on your computer at the same time — a React app, a backend server, a database.

So when a request arrives at your computer, how does the computer know **who it is meant for**? A **port** solves exactly this confusion.

IP address

The address of the building. It identifies the whole computer.

Port

The flat number inside that building. It identifies **one specific program** on that computer.

`http://localhost:4567` literally means: go to **this same computer**, and knock on the door of **flat number 4567**.

- `localhost` means your own computer (its IP is `127.0.0.1`)
- `4567` is the port number

Port numbers range from **0 to 65535**. Some of them are already reserved:

PORT	USED BY
80	Default for HTTP
443	Default for HTTPS
27017	MongoDB
3000 / 5000 / 8080	Development servers (just by convention)

Two important points

1. Ports 0 to 1023 are reserved for the system, so do not touch them. For your own projects, pick any number **above 3000**.

2. Since 80 and 443 are the defaults, you never have to type a port when visiting `https://google.com` — the browser adds `:443` for you automatically.

An error you will definitely see

`EADDRINUSE: address already in use`

It means somebody already lives in that flat. Either close the old terminal (`Ctrl + C`) or use a different port.

05 What does `listen()` actually do?

Simply **creating** a server does nothing on its own. Opening a shop is not enough — you also have to raise the shutter.

`server.listen(4567)` means:

> *"I am now sitting at the door numbered 4567. From this moment on, I will keep listening for anyone who rings the bell."*

One thing you should notice

Once `listen()` runs, the program **does not exit**. A normal JavaScript file prints something and quits immediately, but a server file keeps running — because it is waiting. To stop it, press **Ctrl + C** in the terminal.

06 Your first server — line by line

```
let http = require("http");

let server = http.createServer((req, res) => {
  console.log("hello i m server");
  res.end("ok mene tumhari baat sun li");
});

server.listen(4567, () => {
  console.log("server is running on port 3000");
});
```

Now let us break down every line

`let http = require("http");`

Node.js already ships with a **built-in module** called `http` — you do not have to install it. `require` means: “fetch that module and give it to me.”

`http.createServer((req, res) => { ... })`

This creates a server object. The function you pass inside it will run **once for every request** that arrives.

`req` (request) holds what the client sent — the URL, method, headers and body.
`res` (response) is your tool for sending an answer back.

`console.log("hello i m server");`

This prints in your **terminal**, **not** in the browser. This is server code, and it runs on the server's computer. This single point confuses beginners more than anything else.

`res.end("ok mene tumhari baat sun li");`

This sends the response and **closes** the conversation. After `end()` you cannot send a response again — try it and you will get an error. One request gets exactly one response.

```
server.listen(4567, () => { ... });
```

Start listening on port 4567. The callback inside runs once the server has started successfully.

Spot the small mistake

The code prints `"server is running on port 3000"`, but the server is actually running on **4567**. The code works, yet the message is lying. Small bugs like this cause big confusion later.

So keep the port in a variable — change it in one place and it changes everywhere:

```
const PORT = 4567;

server.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Run it and test it

1. Save the file as `server.js`
2. Run it in the terminal:

```
node server.js
```

3. Open `http://localhost:4567` in your browser

Now try this experiment

Open `/`, then `/users`, then `/anything-random`.

You get the same answer everywhere. That is because we never checked which URL the request came to. And that is exactly where the next topic begins.

07 What is the problem with the raw `http` module?

In a real application, different URLs need to do different things. If you try that with the raw `http` module, your code starts to look like this:

```
const http = require("http");

const server = http.createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") {
    res.end("Home page");

  } else if (req.url === "/users" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "application/json" });
    res.end(JSON.stringify(users));

  } else if (req.url === "/users" && req.method === "POST") {
    let raw = "";
    req.on("data", (chunk) => { raw += chunk; });
    req.on("end", () => {
      const body = JSON.parse(raw);      // parse it yourself
      users.push(body);
      res.writeHead(201);
      res.end("User created");
    });

  } else {
    res.writeHead(404);
    res.end("Not found");
  }
});
```

Now look at everything that is wrong here:

1. You have to write routing by hand. One `if-else` branch per route. Fifty routes means fifty branches, and the file becomes unreadable.

2. You have to parse the body yourself. When the client sends JSON, it arrives in small pieces called **chunks**. You must join them yourself and then call `JSON.parse` yourself.

3. There is no support for dynamic URLs. To pull the `5` out of `/users/5`, you have to slice the string manually.

4. Every small thing is manual. Set the headers, set the status code, call `JSON.stringify` — every single time.

5. There is no concept of middleware. Something common like a login check has to be copy-pasted into every route.

Every backend developer had to do all of this. So eventually somebody thought: “Let us write it properly once, and everybody can reuse it.”

That is how **Express** was born.

08 What is Express.js?

Express is a **framework** built **on top of** Node's own `http` module.

> *Express does nothing magical. Internally it is still the same `http` module. Express simply makes it so convenient that work which used to take 100 lines now takes 10.*

Raw `http`

You are kneading the dough, rolling it out, and cooking it on the pan yourself.

Express

A roti-maker machine. The result is the same — the effort is far less.

What Express gives you

- **Clean routing** — `app.get()`, `app.post()`, no more if-else chains
- **Dynamic routes** — `/users/:id` works out of the box
- **A middleware system** — common work written once, in one place
- **Ready-made helpers** — `res.json()`, `res.status()`, `res.send()`
- **Body parsing in a single line** — `express.json()`

How to install it

```
npm init -y
npm install express
```

Why didn't we install `http`, but we do install Express?

`http` comes **built into** Node itself. Express is a **third-party** package written by someone else, so it has to be downloaded with `npm install`. Once installed, it lands in the `node_modules` folder and its name gets written into `package.json`.

09 The same server in Express

```
const express = require("express");
const app = express();

const PORT = 4567;

app.get("/", (req, res) => {
  res.send("Hello from Express");
});

app.get("/about", (req, res) => {
  res.send("This is the about page");
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Line by line

`const express = require("express");`

Pull in the package — exactly the way we pulled in `http` earlier.

`const app = express();`

Calling `express()` gives you an `app` object. **This `app` is your entire server.** Everything from here on happens on it.

`app.get("/", (req, res) => { ... });`

“When someone comes to `/` using the **GET** method, run this function.” That is it. This is the same thing you had to express as `if (req.url === "/" && req.method === "GET")` in raw http.

`res.send("Hello from Express");`

Send the response. Express sets the content-type for you — send a string and it is treated as HTML, send an object and it becomes JSON. In raw http you had to do all of that yourself.

`app.listen(PORT, () => { ... });`

Exactly the same job that `server.listen()` was doing earlier. Internally Express still calls `http.createServer(app)` — you just do not see it.

Useful methods on `res`

METHOD	WHAT IT DOES
`res.send("hi")`	Send anything — a string, object or array
`res.json({ a: 1 })`	Send JSON — **use this when building an API**
`res.status(404)`	Set the status code (it sends nothing by itself, you must chain onto it)
`res.status(201).json({...})`	Both together — **the most common pattern**
`res.end()`	Close the response without sending any data

10 What is CRUD?

Every application in the world basically performs only **four** operations — whether it is Instagram, a food delivery app, or your college portal.

LETTER	OPERATION	HTTP METHOD	EXAMPLE
C	Create — make something new	`POST`	A new user signs up
R	Read — view something	`GET`	Opening a profile
U	Update — change something	`PUT` / `PATCH`	Editing your bio
D	Delete — remove something	`DELETE`	Deleting an account

PUT vs PATCH

PUT replaces the whole thing. **PATCH** changes only the fields you sent.

In the beginning, just use **PUT** — it keeps things simple. You can pick up PATCH later.

Never put the action inside the URL

The **method** describes the action, not the URL.

Wrong: `/getUsers` · `/createUser` · `/deleteUser`

Right: `GET /users` · `POST /users` · `DELETE /users/5`

A URL should name the **thing** (a noun), never the **action** (a verb). And the thing is always written in plural — `/users`, `/products`.

11 A complete CRUD API with user data

We have not covered databases yet, so we will keep the users in a simple **array**. Restarting the server will wipe the data — that is expected, and MongoDB will fix it later.

Step 1 — setup and data

```
const express = require("express");
const app = express();

app.use(express.json());     // needed to read the body - keep it ABOVE routes

let users = [
  { id: 1, name: "Aarav", age: 19, city: "Indore" },
  { id: 2, name: "Priya", age: 20, city: "Bhopal" },
];
```

Step 2 — the two READ routes (GET)

```
// ---------- READ : all users ----------
app.get("/users", (req, res) => {
  res.status(200).json(users);
});

// ---------- READ : one user ----------
app.get("/users/:id", (req, res) => {
  const id = Number(req.params.id);            // string -> number
  const user = users.find((u) => u.id === id);

  if (!user) {
    return res.status(404).json({ message: "User not found" });
  }
  res.status(200).json(user);
});
```

Step 3 — CREATE (POST)

```
// ---------- CREATE ----------
app.post("/users", (req, res) => {
  const { name, age, city } = req.body;        // data arrives in the body

  if (!name || !age) {
    return res.status(400).json({ message: "name and age are required" });
  }

  const newUser = { id: users.length + 1, name, age, city };
  users.push(newUser);

  res.status(201).json(newUser);               // 201 = Created
});
```

Step 4 — UPDATE and DELETE

```
// ---------- UPDATE ----------
app.put("/users/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = users.findIndex((u) => u.id === id);

  if (index === -1) {
    return res.status(404).json({ message: "User not found" });
  }

  users[index] = { ...users[index], ...req.body, id };
  res.status(200).json(users[index]);
});

// ---------- DELETE ----------
app.delete("/users/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = users.findIndex((u) => u.id === id);

  if (index === -1) {
    return res.status(404).json({ message: "User not found" });
  }

  const removed = users.splice(index, 1);
  res.status(200).json({ message: "Deleted", user: removed[0] });
});

app.listen(4567, () => console.log("Server ready on 4567"));
```

Now let us go route by route

> `GET /users`
>
> Returns the list of all users. No input is needed, so we simply send the array back with status `200`.

> `GET /users/:id`
>
> Returns one specific user. Here `:id` is a **dynamic part**. If you hit `/users/2`, then `req.params.id` will contain `"2"`.
>
> **Pay attention:** `"2"` is a **string**, not a number. That is why we wrap it in `Number()`. If you skip that, `"2" === 2` evaluates to false and the user is never found. **This is the number one beginner bug.**

`POST /users`

Creates a new user. The data does not come through the URL — it arrives in the **body**, which is why we read it from `req.body`.

When something is created, send status `201 Created`, not `200`.

`PUT /users/:id`

Updates a user. First check whether the user exists at all, then spread the new data on top of the old data.

Notice that `id` is written again at the end — this stops the client from changing the id, accidentally or deliberately.

`DELETE /users/:id`

Removes a user. `splice` takes the element out of the array and returns an array of whatever it removed.

Notice that every route follows the same pattern

1. Extract the input (params / body / query)
2. Check whether it is valid
3. Do the actual work
4. Send a response with the correct status code

This pattern repeats in every API you will ever write. Understand it once and the rest becomes easy.

Why did we write `return`?

If you do not put `return` in front of `res.status(404).json(...)`, the code below it will run **as well**, and the server will try to send a second response.

Then you get this error: `Cannot set headers after they are sent to the client`

Remember: **one request gets exactly one response.**

12 What is middleware?

Middleware is a function that runs **in between** — after the request arrives from the client, but **before** it reaches your route.

The college gate analogy

You walk in through the college gate. Before you reach your classroom:

1. The guard checks your ID card → **2.** Your entry is written in the register → **3.** Then you enter the classroom

The guard and the register are the **middleware**. The classroom is your **route handler**.

Every middleware receives three things:

```
app.use((req, res, next) => {
  console.log(req.method, req.url, new Date().toLocaleTimeString());
  next();          // <- forget this and the request hangs forever
});
```

- `req` — the request. You can also modify it, for example by attaching `req.user`
- `res` — the response. You can even send a response from here and stop the request early
- `next()` — "I am done, pass it along"

The most common mistake

Forgetting to call `next()` means the request **hangs forever**. The browser keeps spinning, nothing happens, and you do not even get an error message. People lose hours debugging this.

13 `express.json()` — the middleware you cannot skip

First, see the problem

```
const app = express();
// express.json() has NOT been added...

app.post("/users", (req, res) => {
  console.log(req.body);      // undefined
  res.send("done");
});
```

`req.body` comes out as **undefined**. Why?

Because the client did send JSON, but on the network it arrives as **raw bytes**, broken into small chunks. By default Express has no idea what to do with that raw data, so it simply ignores it.

Now the solution

```
const express = require("express");
const app = express();

app.use(express.json());        // <- this single line does all the work

app.post("/users", (req, res) => {
  console.log(req.body);        // { name: 'Aarav', age: 19 }
  res.status(201).json(req.body);
});
```

Internally, that one line does four things:

1. Joins all the incoming chunks of the request together
2. Checks whether the header says `Content-Type: application/json`
3. If it does, runs `JSON.parse` on the text
4. Puts the result into `req.body`

Now `req.body` hands you a ready JavaScript object — the same work that took eight lines in raw http.

> **Where should you write it?**
>
> **Always above your routes.** Middleware runs top to bottom. If you write it below the routes, the routes run first and `req.body` will be undefined all over again.

It has a sibling too

```
app.use(express.json());                          // for JSON
app.use(express.urlencoded({ extended: true }));  // for HTML forms
```

`express.urlencoded()` handles data coming from an HTML `<form>`, which arrives in the `name=Aarav&age=19` format.

Simple rule: **`express.json()` for JSON, `express.urlencoded()` for HTML forms.** Write both — there is no harm.

> **When testing in Postman**
>
> Go to Body → **raw** → and select **JSON** from the dropdown. If **Text** stays selected, `req.body` will be empty again and you will end up blaming `express.json()` for an hour.

14 Every way of receiving data in an API

Learn this table thoroughly. It comes up in interviews, and you will use it every single day.

SOURCE	VISIBLE IN URL?	USED WITH	USED FOR
`req.params`	Yes	All methods	**Which** item to act on — the ID
`req.query`	Yes	Mostly GET	Filtering, search, sorting, pagination
`req.body`	**No**	POST, PUT, PATCH	The actual data — forms, JSON
`req.headers`	**No**	All methods	Tokens, content-type, meta information
`req.file` / `req.files`	**No**	POST, PUT	Uploading images, PDFs, videos

1. `req.params` — part of the URL itself

Used when you create a dynamic segment in the URL with a colon.

```
// GET /users/7
app.get("/users/:id", (req, res) => {
  console.log(req.params);      // { id: '7' }
  console.log(req.params.id);   // '7'  <- this is a STRING, not a number
  const id = Number(req.params.id);
});

// More than one dynamic part
// GET /users/7/posts/3
app.get("/users/:userId/posts/:postId", (req, res) => {
  console.log(req.params);      // { userId: '7', postId: '3' }
});
```

When to use it: when you need to point at **one particular** item.

2. `req.query` — whatever comes after the `?`

In the URL, you write `key=value` after a `?` and join multiple pairs with `&`.

```
// GET /users?city=Indore&limit=2
app.get("/users", (req, res) => {
  console.log(req.query);       // { city: 'Indore', limit: '2' }

  const { city, limit } = req.query;

  let result = users;
  if (city)  result = result.filter((u) => u.city === city);
  if (limit) result = result.slice(0, Number(limit));

  res.json(result);
});
```

Keep in mind

Query values are also **always strings**. Sending `limit=2` gives you `"2"`, not `2`. Convert it with `Number()` whenever you need a number.

Params vs query — confusion cleared

`/users/7` → "give me **this specific** user" — that is **params**

`/users?city=Indore` → "give me users **like this**" — that is **query**

Params **identify**. Query **filters**.

3. `req.body` — the hidden data

It never appears in the URL; it is tucked inside the request. That is exactly why passwords are always sent in the body and **never** in the query — otherwise they would sit in plain text inside browser history and server logs.

```
app.use(express.json());

app.post("/login", (req, res) => {
  const { email, password } = req.body;   // never visible in the URL
  res.json({ message: "Login attempt for " + email });
});
```

We do not send a body with a GET request. It is technically possible, but nobody does it in practice.

4. `req.headers` — extra information

```
app.get("/profile", (req, res) => {
  console.log(req.headers.authorization);     // "Bearer abc123..."
  console.log(req.headers["content-type"]);   // "application/json"
  res.send("ok");
});
```

This is where the login token arrives. One thing worth noting: header names always reach you in **lowercase**, no matter what casing the client used.

5. `req.file` / `req.files` — file uploads

Files cannot be sent inside JSON. They need a different format called `multipart/form-data`. And to understand that format, Express needs a separate package: **multer**.

```
npm install multer
```

```
const multer = require("multer");
const upload = multer({ dest: "uploads/" });

// ONE file
app.post("/avatar", upload.single("photo"), (req, res) => {
  console.log(req.file);
  // { originalname, mimetype, size, filename, path }
  res.json({ message: "Uploaded", file: req.file.filename });
});

// MULTIPLE files (max 5)
app.post("/gallery", upload.array("photos", 5), (req, res) => {
  console.log(req.files);       // array of files
  res.json({ count: req.files.length });
});
```

- One file → `req.file` (singular)
- Multiple files → `req.files` (plural, an array)
- If you send normal text fields alongside the file, those still arrive in `req.body`

> **Notice something**
>
> `multer` is itself a **middleware** — look at where it sits, right between the route path and the handler function. It is exactly the concept you just learned above.

15 Mistakes everybody makes

1. Writing `express.json()` **below** the routes → `req.body` is undefined

2. Treating `req.params.id` as a number → the item is never found

3. Forgetting `return` before `res.json()` → “Cannot set headers after they are sent”

4. Forgetting `next()` inside a middleware → the request hangs forever

5. Getting `EADDRINUSE` → close the old terminal or change the port

6. Selecting Text instead of JSON in Postman → the body arrives empty

7. Changing the code but not restarting the server → the old code keeps running. Run `npm install -g nodemon` and start it with `nodemon server.js` so it restarts on its own.

16 Quick revision

The **client** asks, the **server** answers

HTTP is the shared language · **HTTPS** is that same language, locked

IP is the building address · **Port** is the flat number

`listen()` means sitting at the door, waiting for the bell

Raw `http` can do everything — but only with a lot of effort

Express does the same work in far less code

CRUD maps to POST · GET · PUT · DELETE

Middleware is the guard standing in between — `next()` is mandatory

`express.json()` turns the raw body into a JavaScript object

Data arrives from five places: **params · query · body · headers · files**

4xx means you messed up · **5xx** means I messed up

17 Practice

1. Build a server using raw `http` where `/` returns “Home” and `/about` returns “About”. Do it with if-else, so you feel the pain.
2. Now build the exact same thing in Express. Compare how long each one took.
3. Build a full CRUD API for **books** with the fields `id`, `title`, `author` and `price`.
4. Make `GET /books?author=Chetan` work — filter using the query.
5. Add validation to `POST /books` — if `title` is missing, respond with `400`.
6. Write a middleware that logs the method, URL and time of every request to the terminal.
7. Hit `GET /books/9999` and confirm that you get a `404`.
8. **Bonus:** build `GET /books?page=1&limit=5` — basic pagination.

> Test everything in **Postman**, not in the browser. A browser can only send GET requests — for POST, PUT and DELETE you need Postman.